

CA-2 Submission Plan — Step-by-Step Execution & Screenshot Guide

Goal: Run the project, collect ~15 numbered screenshots, write the 1500-word report, record the screencast, zip and upload. **Deadline:** Sunday 24th May @ 23:59

PHASE 1: Start the Project (5 minutes)

Open **PowerShell as Administrator** and `cd` to your project folder:

```
cd "c:\Users\bhara\Desktop\Final\Chinni"
```

Step 1.1 — Bootstrap the entire project

```
powershell -ExecutionPolicy Bypass -File "scripts\bootstrap-from-scratch.ps1"
```

Wait for it to finish. You should see 7 Passed, 0 Failed at the end.

Step 1.2 — Start port-forwards (open 3 separate PowerShell terminals)

Terminal 2:

```
kubectl port-forward svc/gateway 8080:80 -n checkout-demo
```

Terminal 3:

```
kubectl port-forward svc/prometheus 9090:9090 -n checkout-demo
```

Terminal 4:

```
kubectl port-forward svc/grafana 3000:3000 -n checkout-demo
```

Keep all 3 terminals open throughout.

PHASE 2: Collect Screenshots (15 screenshots)

Take each screenshot and save it with the filename shown. These map directly to the report sections.

❑ Screenshot 1 — pods-running.png

What: All pods running in the cluster **Command:**

```
kubectl get pods -n checkout-demo
```

What to capture: The terminal output showing all 7 pods (gateway, checkout, pricing, inventory, postgres, prometheus, grafana) in **Running** status with 1/1 **READY**. **Report section:** Part 1 (Secure Design) — proves the system is deployed and healthy.

❑ Screenshot 2 — non-root-user.png

What: Proof containers run as non-root **Command** (run all 4):

```
kubectl exec deploy/gateway -n checkout-demo -- whoami
kubectl exec deploy/checkout -n checkout-demo -- whoami
kubectl exec deploy/pricing -n checkout-demo -- whoami
kubectl exec deploy/inventory -n checkout-demo -- whoami
```

What to capture: All 4 commands returning appuser. **Report section:** Part 1 (Secure Design) — proves container workload hardening.

❏ Screenshot 3 — network-policies.png

What: All 5 NetworkPolicies deployed **Command:**

```
kubectl get networkpolicies -n checkout-demo
```

What to capture: The table showing 5 policies (gateway-netpol, checkout-netpol, pricing-netpol, inventory-netpol, postgres-netpol). **Report section:** Part 1 (Secure Design) — proves network isolation is active.

❏ Screenshot 4 — network-policy-detail.png

What: One detailed NetworkPolicy to show the actual rules **Command:**

```
kubectl describe networkpolicy checkout-netpol -n checkout-demo
```

What to capture: The full describe output showing ingress/egress rules. **Report section:** Part 1 (Secure Design) — shows a real policy configuration with allowed ports and pod selectors.

❏ Screenshot 5 — checkout-api-response.png

What: A successful checkout API call **Command:**

```
Invoke-RestMethod -Uri http://localhost:8080/api/checkout -Method POST `
-ContentType "application/json" `
-Headers @{"X-Request-Id"="screenshot-test-001"} `
-Body '{"items":[{"item_id":"item-1","quantity":2}]}'
```

What to capture: The JSON response showing total_cost: 20.0, order_id, and request_id. **Report section:** Part 2 (Observability) — proves the system processes requests end-to-end.

❏ Screenshot 6 — log-correlation.png

What: Request ID correlation in logs across services **Commands:**

```
kubectl logs deploy/gateway -n checkout-demo --tail=10
kubectl logs deploy/checkout -n checkout-demo --tail=10
```

What to capture: Both log outputs showing the same request_id=screenshot-test-001 appearing in gateway AND checkout logs. **Report section:** Part 2 (Observability) — proves distributed log correlation works.

❏ Screenshot 7 — prometheus-targets.png

What: Prometheus targets page showing all services are UP **Action:** Open browser → go to <http://localhost:9090/targets> (<http://localhost:9090/targets>). **What to capture:** The Prometheus targets page showing gateway, checkout, pricing, inventory all with status UP (green). **Report section:** Part 2 (Observability) — proves Prometheus is scraping all services.

❏ Screenshot 8 — prometheus-metrics-query.png

What: A Prometheus query showing actual collected metrics **Action:** Open browser → go to <http://localhost:9090/graph> (<http://localhost:9090/graph>) → paste this query:

```
rate(http_requests_total[1m])
```

Click **Execute** → switch to the **Graph** tab. **What to capture:** The graph showing request rate data points. **Report section:** Part 2 (Observability) — proves metrics are being collected.

❑ Screenshot 9 — grafana-dashboard.png

What: The Grafana RED metrics dashboard **Action:** Open browser → go to <http://localhost:3000> (<http://localhost:3000>) → Login with admin / admin → Skip password change → Navigate to **Dashboards** → **Chinni** folder → **Chinni Checkout System RED Metrics** **What to capture:** The full dashboard showing all 3 panels: Request Rate (RPS), Error Rate, P95 Latency. **Report section:** Part 2 (Observability) — THE key screenshot proving monitoring visualization.

[!TIP] Before taking this screenshot, send 10-20 checkout requests to generate visible graph data:

```
1..20 | ForEach-Object { try { Invoke-RestMethod -Uri http://localhost:8080/api/checkout -Method POST -ContentType "application/json" -Body '{"id":1,"amount":100}' } catch { } }
```

❑ Screenshot 10 — grafana-datasource.png

What: Grafana showing Prometheus as a connected datasource **Action:** In Grafana → go to **Connections** → **Data sources** → click **Prometheus** **What to capture:** The datasource config page showing <http://prometheus:9090> and a green "Successfully queried the Prometheus API" message. **Report section:** Part 2 (Observability) — proves Grafana-Prometheus integration.

❑ Screenshot 11 — trivy-scan-output.png

What: Trivy vulnerability scan results **Action:** Open the file `trivy-gateway-report.txt` in your text editor (or run):

```
Get-Content .\trivy-gateway-report.txt | Select-Object -First 50
```

What to capture: The top portion of the Trivy report showing the vulnerability table with severity levels (HIGH/CRITICAL). **Report section:** Part 3 (Security Testing) — proves image scanning was performed.

❑ Screenshot 12 — kubescape-score.png

What: Kubescape compliance score **Action:** Open the file `kubescape-report.txt` in your text editor (or run):

```
Get-Content .\kubescape-report.txt
```

What to capture: The output showing the overall compliance score of **84%** and the control pass/fail summary. **Report section:** Part 3 (Security Testing) — proves posture scanning was performed.

❑ Screenshot 13 — kubescape-pdf.png

What: The Kubescape PDF report **Action:** Double-click `kubescape-report.pdf` to open it in your PDF viewer. **What to capture:** The first page of the PDF showing the visual compliance summary. **Report section:** Part 3 (Security Testing) — alternative visual evidence of posture review.

❑ Screenshot 14 — verify-all-pass.png

What: The automated verification suite showing 7/7 PASS **Command:**

```
powershell -ExecutionPolicy Bypass -File "scripts\verify-all.ps1"
```

What to capture: The full terminal output showing all 7 checks with [PASS] and the summary line 7 Passed, 0 Failed. **Report section:** Part 3 (Security Testing) — proves everything works together.

❑ Screenshot 15 — dockerfile-security.png

What: One Dockerfile showing the non-root hardening **Action:** Open `services/gateway/Dockerfile` in your editor. **What to capture:** The full Dockerfile showing `RUN useradd,USER appuser,RUN chown`. **Report section:** Part 1 (Secure Design) — shows the actual code change for container hardening.

PHASE 3: Write the PDF Report (1500 words max)

Use this outline — paste the relevant screenshots into each section:

Report Outline

Title Page

- **Title:** ETP Design CA-2 — Security & Observability Review
- **Module:** Enterprise Architecture Design (ENTPH6001)
- **Student Name:** [Your Name]
- **Student ID:** [Your ID]
- **Date:** 24 May 2026

Section 1: Secure Design Review & Architecture (Part 1 — 25%) (~400 words)

1.1 Risks Identified in CA-1 System Write 3-4 bullet points about risks in the original system:

- Containers ran as `root` → risk of node compromise
- No `NetworkPolicies` → any pod could talk to any pod (lateral movement risk)
- No image scanning → unknown vulnerabilities in dependencies
- Database credentials embedded in deployment YAML

1.2 Mitigations Applied

- Non-root Dockerfiles (`USER appuser`) — **insert Screenshot 2 + Screenshot 15**
- 5 `NetworkPolicies` enforcing least-privilege — **insert Screenshot 3 + Screenshot 4**
- Kubernetes Secrets for credentials

1.3 Architecture Diagrams

Diagram A — Current State (Before CA-2): Draw a simple diagram showing: Browser → Gateway → Checkout → Pricing/Inventory/Postgres. Label it "No boundaries — all pods can communicate freely, containers run as root".

Diagram B — Improved State (After CA-2): Draw the same flow but add:

- `NetworkPolicy` firewall boundaries between each service
- Prometheus scraping arrows to `/metrics` endpoints
- Grafana connected to Prometheus
- "Non-root" labels on all service containers
- Trivy/Kubescape scanning boundaries on the build pipeline

[!TIP] Use [draw.io \(https://app.diagrams.net/\)](https://app.diagrams.net/) to create both diagrams. Export as PNG and paste into the report.

Section 2: Logging, Monitoring & Observability (Part 2 — 25%) (~350 words)

2.1 What was added

- `prometheus-fastapi-instrumentator` added to all 4 services, exposing `/metrics`
- Prometheus deployed to scrape all targets every 15 seconds
- Grafana deployed with auto-provisioned RED metrics dashboard

2.2 Evidence

- **insert Screenshot 7** (Prometheus targets all UP)
- **insert Screenshot 9** (Grafana RED dashboard)
- **insert Screenshot 10** (Grafana-Prometheus datasource)

2.3 Realistic Observability Scenario Describe how you would diagnose a checkout failure:

1. Grafana Error Rate panel spikes from 0% to 10%
 2. Check logs with request ID: `kubectl logs deploy/checkout -n checkout-demo | Select-String "request_id=abc"`
 3. Discover pricing service returned 500
 4. Correlate using `X-Request-Id` header propagated across Gateway → Checkout → Pricing
- **insert Screenshot 5** (successful API response with `request_id`)
 - **insert Screenshot 6** (log correlation)

Section 3: Security Testing (Part 3 — 30%) (~500 words)

3.1 Vulnerability Scanning — Trivy

- Scanned all 4 microservice images
- Found 4 HIGH severity CVEs (in `libncursesw6` and Python package dependencies)
- **insert Screenshot 11** (Trivy report)

3.2 Configuration Posture Review — Kubescape

- Scanned all K8s manifests under `k8s/`
- Overall compliance score: **84%**
- Privileged container control: PASS
- Host network access: PASS
- Non-root containers: PASS
- **insert Screenshot 12 or 13** (Kubescape score/PDF)

3.3 Deeper Test — Runtime Verification

- Verified non-root execution at runtime (`kubectl exec -- whoami`)
- Verified NetworkPolicy enforcement blocks unauthorised traffic
- Ran automated 7-check validation suite
- **insert Screenshot 14** (7/7 PASS)

3.4 Findings Prioritisation

Priority	Finding	Status	Justification
☐ Critical	Containers running as root	☐ Fixed	Direct node compromise risk
☐ High	No network isolation	☐ Fixed	Lateral movement enables full cluster takeover
☐ Medium	4 HIGH CVEs in base image	☐ Recommended	Upgrade to <code>python:3.12-slim</code>
☐ Low	No CPU/memory limits	☐ Recommended	Prevents resource exhaustion / DoS

3.5 What would change next

- Upgrade Python base image to `3.12-slim`
- Add resource limits to all deployments
- Implement mTLS between services
- Add OPA Gatekeeper admission policies

Section 4: Conclusion (~100 words)

Summarise that the system has been hardened, monitored, and tested. All 7 verification checks pass.

- **insert Screenshot 1** (all pods running)
- **insert Screenshot 14** (7/7 PASS)

PHASE 4: Record Screencast (5–10 minutes)

Use **OBS Studio** or **Windows Game Bar** (`Win + G`) to record your screen.

Screencast Script:

Time	What to show	What to say
0:00–0:30	Show project folder structure	"This is the Chinni checkout system project for CA-2..."
0:30–1:30	Run <code>kubectl get pods -n checkout-demo</code>	"All 7 pods are running. Let me show you the security hardening..."
1:30–2:30	Run the <code>whoami</code> commands on all 4 services	"All containers run as non-root user 'appuser', not root..."
2:30–3:30	Run <code>kubectl get networkpolicies -n checkout-demo</code>	"We have 5 NetworkPolicies enforcing least-privilege communication..."
3:30–4:30	Show Grafana dashboard in browser (<code>http://localhost:3000</code>)	"Here is the Grafana RED metrics dashboard showing Request Rate, Error Rate, and P95 Latency..."
4:30–5:30	Send a checkout request and show it appear on the dashboard	"I'll send a live request and you can see the metrics update in real-time..."

Time	What to show	What to say
5:30–6:30	Show Prometheus targets page (http://localhost:9090/targets)	"Prometheus is scraping all 4 microservices. All targets are UP..."
6:30–7:30	Show Trivy report + Kubescape PDF	"We performed vulnerability scanning with Trivy and posture scanning with Kubescape. The compliance score is 84%..."
7:30–8:30	Run <code>verify-all.ps1</code>	"Finally, our automated verification suite confirms 7 out of 7 checks pass..."
8:30–9:00	Show the Dockerfile with non-root user	"The key security changes include non-root users in all Dockerfiles..."
9:00–10:00	Wrap up	"In summary, we've hardened the system with non-root containers and network policies, added full observability with Prometheus and Grafana, and validated everything with security scans and automated tests."

After recording:

1. Upload the video to your **student OneDrive**
2. Create a **shareable link** (Anyone with the link can view)
3. Paste the link in your report or Brightspace submission

PHASE 5: Submit

- ☐ Put all screenshots in a `screenshots/` folder inside the project
- ☐ Save the PDF report as `CA2-Report.pdf` in the project root
- ☐ **Delete the large binary files** before zipping to reduce size:

```
Remove-Item .\trivy.exe -Force
Remove-Item .\kubescape.exe -Force
Remove-Item .\kubescape_v2.exe -Force
Remove-Item .\kubescape_v3.exe -Force
Remove-Item -Recurse .\.venv -Force
Remove-Item -Recurse .\pytest_cache -Force
```

- ☐ Zip the folder: Right-click Chinni folder → **Send to** → **Compressed (zipped) folder**
- ☐ Upload to **Brightspace** before **Sunday 24th May @ 23:59**